

Performans ve optimizasyon

Bu belge, Django kodunuzun daha verimli, daha hızlı ve daha az sistem kaynağı kullanarak çalışmasını sağlayacak teknik ve araçlara genel bir bakış sunmaktadır.

Giriş

Genellikle ilk öncelik, beklenen çıktıyı üretmek için gereken mantığı yerine getiren, düzgün çalışan bir kod yazmaktır. Ancak bazen bu, kodun istenen verimlilikte çalışması için yeterli olmaz.

Bu durumda, kodun davranışını hiç etkilemeden ya da en az düzeyde etkileyerek performansını artıracak bir şeye — pratikte genellikle bir dizi önlemeye — ihtiyaç duyulur.

Genel Yaklaşımlar

Ne için optimizasyon yapıyorsunuz?

"Performans" derken neyi kastettiğinizi net bir şekilde belirlemek önemlidir. Performansı ölçen tek bir gösterge yoktur.

Hızın artırılması bir program için en bariz hedef olabilir, ancak bazen daha az bellek kullanımı ya da veritabanı veya ağ üzerindeki yükün azaltılması gibi başka performans iyileştirmeleri de hedeflenebilir.

Bir alandaki iyileştirmeler genellikle başka bir alanda da performans artışı sağlar, ancak bu her zaman böyle değildir; bazen bir iyileştirme, diğerinin aleyhine olabilir. Örneğin, bir programın hızındaki bir iyileştirme, programın daha fazla bellek kullanmasına neden olabilir. Daha da kötüsü, bu durum kendi amacına ters düşebilir - hız iyileştirmesi o kadar fazla bellek tüketirse ve sistemin belleği yetersiz kalırsa, yarardan çok zarar vermiş olursunuz.

Akılda tutulması gereken başka ödünleşmeler de vardır. Kendi zamanınız, CPU zamanından daha değerli bir kaynaktır. Bazı iyileştirmeler, uygulanmaya değmeyecek kadar zor olabilir veya kodun taşınabilirliğini veya bakımını etkileyebilir. Her performans iyileştirmesi çabaya değer değildir.

Bu nedenle, hangi performans iyileştirmelerini hedeflediğinizi bilmeniz ve bu yönde ilerlemek için iyi bir nedeniniz olduğunu bilmeniz gerekir; bunun için de şunlara ihtiyacınız vardır:

Performans kıyaslaması

Kodunuzdaki verimsizliklerin nerede olduğunu sadece tahmin etmek ya da varsaymak yeterli değildir.

Django araçları

django-debug-toolbar, kodunuzun ne yaptığını ve bu işlemleri gerçekleştirmek için ne kadar zaman harcadığını gösteren oldukça kullanışlı bir araçtır. Özellikle, sayfanızın oluşturduğu tüm SQL sorgularını ve her birinin ne kadar sürdüğünü size gösterebilir.

Araç çubuğu için, örneğin önbellek performansı ve şablon işleme süreleri hakkında raporlar sunabilen üçüncü taraf paneller de mevcuttur.

Üçüncü taraf hizmetleri

Sitenizin sayfalarının performansını uzaktaki bir HTTP istemcisinin bakış açısıyla analiz edip raporlayan ve böylece gerçek bir kullanıcının deneyimini simüle eden birçok ücretsiz hizmet bulunmaktadır.

Bu hizmetler kodunuzun iç işleyişi hakkında rapor veremez, ancak Django ortamının içinden yeterince ölçülemeyen unsurlar da dahil olmak üzere sitenizin genel performansı hakkında yararlı bilgiler sağlayabilir.

Benzer bir analiz gerçekleştiren birkaç ücretli hizmet de mevcuttur; bunlardan bazıları Django'yu tanır ve kod tabanınızla entegre olarak performansını çok daha kapsamlı bir şekilde profilleyebilir.

İşleri en başından doğru yapın

Optimizasyon çalışmalarının bir kısmı performans eksikliklerini gidermeyi içerir; ancak bu çalışmaların bir kısmı, performans iyileştirmeyi düşünmeye başlamadan önce bile benimsemeniz gereken iyi uygulamaların bir parçası olarak, zaten yapacağınız işlerin içine entegre edilebilir.

Bu açıdan Python, çalışmak için mükemmel bir dildir; çünkü zarif görünen ve doğru hissettiren çözümler genellikle en iyi performansı gösteren çözümlerdir. Çoğu beceride olduğu gibi, neyin “doğru göründüğünü” öğrenmek pratik gerektirir; ancak en yararlı kılavuzlardan biri şudur:

Uygun seviyede çalışın

Django, işleri halletmek için pek çok farklı yol sunar; ancak bir şeyi belirli bir şekilde yapmanın mümkün olması, bunun en uygun yol olduğu anlamına gelmez. Örneğin, aynı şeyi – örneğin bir koleksiyondaki öğe sayısını – bir **QuerySet**’te, Python’da veya bir şablonda hesaplayabileceğinizi fark edebilirsiniz.

Ancak, bu işi daha yüksek seviyelerde yapmak yerine daha düşük seviyelerde yapmak neredeyse her zaman daha hızlı olacaktır. Daha yüksek seviyelerde sistem, nesneleri çok sayıda soyutlama seviyesi ve mekanizma katmanı aracılığıyla işlemek zorundadır.

Yani, veritabanı genellikle Python’dan daha hızlı işlem yapabilir; Python ise şablon dilinden daha hızlı işlem yapabilir:

```
# Veritabanında QuerySet işlemi
# hızlıdır, çünkü veritabanları bu konuda çok iyidir

my_bicycles.count()

# Python nesnelerini sayma
# daha yavaş, çünkü yine de bir veritabanı sorgusu ve
# Python nesnelerinin işlenmesi gerekiyor

len(my_bicycles)
```

```
<!--
Django şablon filtresi
Daha da yavaştır, çünkü bunları yine de Python’da saymak zorunda
kalacaktır, ve şablon dilinin getirdiği ek yük nedeniyle

-->
{{ my_bicycles|length }}
```

Genel olarak, bir iş için en uygun seviye, kod yazarken rahat hissedilen en düşük seviyedir.

Not

Yukarıdaki örnek yalnızca açıklayıcı niteliktedir.

İlk olarak, gerçek hayattaki bir durumda, o bağlamda en uygun yöntemi belirlemek için sayım işleminden önce ve sonra neler olduğunu göz önünde bulundurmanız gerekir. Veritabanı optimizasyonu belgesinde, şablon içinde sayım yapmanın daha uygun olacağı bir durum açıklanmaktadır.

İkinci olarak, dikkate alınması gereken başka seçenekler de vardır: gerçek hayattaki bir durumda, şablondan doğrudan **QuerySet.count()** yöntemini çağıran **{{ my_bicycles.count }}** en uygun seçim olabilir.

Önbellekleme

Bir değeri hesaplamak genellikle maliyetli (yani, çok kaynak tüketen ve yavaş) bir işlemdir; bu nedenle, bu değeri hızlı erişilebilir bir önbelleğe kaydederek bir sonraki kullanım için hazır tutmak büyük fayda sağlayabilir.

Bu teknik o kadar önemli ve güçlüdür ki, Django kapsamlı bir önbellekleme çerçevesinin yanı sıra daha küçük ölçekli önbellekleme işlevleri de içermektedir.

Önbelleğe alma çerçevesi

Django'nun önbellekleme çerçevesi, dinamik içeriği her istek için yeniden hesaplanmasına gerek kalmayacak şekilde kaydederek performans artışı için çok önemli fırsatlar sunar.

Kullanım kolaylığı açısından Django, farklı düzeylerde önbellekleme seçenekleri sunar: belirli görünümünün çıktısını, yalnızca oluşturulması zor parçaları veya hatta tüm siteyi önbelleğe alabilirsiniz.

Önbelleklemeyi, kötü yazıldığı için düşük performans gösteren kodu iyileştirmenin bir alternatifi olarak görmemelisiniz. Bu, iyi performans gösteren kod üretmeye yönelik son adımlardan biridir, bir kısayol değildir.

cached_property

Bir sınıf örneğinin yöntemini birden fazla kez çağırmak sık karşılaşılan bir durumdur. Eğer bu işlem kaynak tüketen bir işlemse, bu durum israf yaratabilir.

cached_property dekoratörünü kullanmak, bir özelliğin döndürdüğü değeri kaydeder; işlem o örnekte bir sonraki sefer çağrıldığında, değeri yeniden hesaplamak yerine kaydedilen değeri döndürür. Bunun yalnızca self'i tek argüman olarak alan yöntemlerde çalıştığını ve yöntemi bir özelliğe dönüştürdüğünü unutmayın.

Bazı Django bileşenlerinin de kendi önbellekleme işlevleri vardır; bunlar, ilgili bileşenlerle ilgili bölümlerde aşağıda ele alınmaktadır.

Tembelliği anlamak

Tembellik, önbelleklemeyi tamamlayıcı bir stratejidir. Önbellekleme, sonuçları kaydederek yeniden hesaplamayı önler; tembellik ise hesaplamayı gerçekten gerekli olana kadar erteler.

Tembellik, nesneler somutlaştırılmadan önce, hatta somutlaştırılmaları mümkün olmadan önce bile onlara başvurmamızı sağlar. Bunun sayısız kullanımı vardır.

Örneğin, tembel çeviri, hedef dil henüz bilinmeden önce bile kullanılabilir; çünkü çevirilen dize, örneğin işlenmiş bir şablonda olduğu gibi, gerçekten gerekli olana kadar gerçekleşmez.

Tembellik, aynı zamanda işi en başından önlemeye çalışarak efor tasarrufu sağlamanın bir yoludur. Yani, tembelliğin bir yönü, yapılması gerekene kadar hiçbir şey yapmamaktır; çünkü sonuçta gerekli olmayabilir. Bu nedenle tembellik, performans üzerinde etkileri olabilir ve ilgili iş ne kadar maliyetliyse, tembellikten elde edilecek kazanç da o kadar fazla olur.

Python, özellikle jeneratör ve jeneratör ifadesi yapıları aracılığıyla tembel değerlendirme için bir dizi araç sağlar. Kodunuzda tembel kalıplardan yararlanma fırsatlarını keşfetmek için Python'daki tembellik konusunu okumaya değer.

Django'da tembellik

Django'nun kendisi oldukça tembeldir. Bunun iyi bir örneği, bir **QuerySet**'in değerlendirilmesinde görülebilir. **QuerySet**'ler tembeldir. Dolayısıyla bir **QuerySet**, tanımladığı öğeleri almak için veritabanına herhangi bir erişim gerektirmeden oluşturulabilir, aktarılabilir ve diğer **QuerySet** örnekleriyle birleştirilebilir. Aktarılan şey, **QuerySet** nesnesidir; veritabanından -sonunda- alınması gereken öğe koleksiyonu değildir.

Öte yandan, belirli işlemler bir **QuerySet**'in değerlendirilmesini zorlar. Bir **QuerySet**'in vaktinden önce değerlendirilmesini önlemek, veritabanına yapılan pahalı ve gereksiz erişimlerden tasarruf sağlayabilir.

Django ayrıca bir **keep_lazy()** dekoratörü sunar. Bu, tembel bir argümanla çağrılan bir fonksiyonun kendisinin de tembel davranmasını ve yalnızca gerektiğinde değerlendirilmesini sağlar. Böylece, maliyetli olabilecek tembel argüman, kesinlikle gerekli olana kadar değerlendirme için çağrılmaz.

Veritabanları

Veritabanı optimizasyonu

Django'nun veritabanı katmanı, geliştiricilerin veritabanlarından en iyi performansı elde etmelerine yardımcı olacak çeşitli yöntemler sunar. Veritabanı optimizasyonu belgeleri, ilgili belgelere yönlendiren bağlantıları bir araya getirir ve veritabanı kullanımınızı optimize etmeye çalışırken atmanız gereken adımları özetleyen çeşitli ipuçları içerir.

Veritabanıyla ilgili diğer ipuçları

Kalıcı bağlantıların etkinleştirilmesi, veritabanı hesaplarına yapılan bağlantıların hızlanmasını sağlar; bu da istek işleme süresinin önemli bir kısmını oluşturur.

Bu, örneğin ağ performansı sınırlı olan sanallaştırılmış sunucularda oldukça faydalıdır.

HTTP performansı

Ara yazılım

Django, sitenizin performansını optimize etmenize yardımcı olabilecek birkaç kullanışlı ara yazılımla birlikte gelir. Bunlar arasında şunlar yer alır:

ConditionalGetMiddleware

ETag ve **Last-Modified** başlıklarına göre koşullu GET yanıtları için modern tarayıcılara destek ekler. Ayrıca gerekirse bir ETag hesaplar ve ayarlar.

GZipMiddleware

Tüm modern tarayıcılar için yanıtları sıkıştırarak bant genişliği ve aktarım süresinden tasarruf sağlar. GZipMiddleware'in şu anda bir güvenlik riski olarak değerlendirildiğini ve TLS/SSL tarafından sağlanan korumayı etkisiz hale getiren saldırılara karşı savunmasız olduğunu unutmayın. Daha fazla bilgi için **GZipMiddleware**'deki uyarıya bakın.

Oturumlar

Önbelleğe alınmış oturumları kullanma

Önbelleğe alınmış oturumları kullanmak, oturum verilerini veritabanı gibi daha yavaş bir depolama kaynağından yükleme ihtiyacını ortadan kaldırarak ve bunun yerine sık kullanılan oturum verilerini bellekte depolayarak performansı artırmanın bir yolu olabilir.

Statik dosyalar

Tanımı gereği dinamik olmayan statik dosyalar, optimizasyon kazançları elde etmek için mükemmel bir hedef oluşturur.

ManifestStaticFilesStorage

Web tarayıcılarının önbellekleme özelliklerinden yararlanarak, ilk indirme işleminden sonra belirli bir dosya için ağ trafiğini tamamen ortadan kaldırabilirsiniz.

ManifestStaticFilesStorage statik dosyaların adlarına içeriğe bağlı bir etiket ekleyerek, tarayıcıların gelecekteki değişiklikleri kaçırmadan bu dosyaları uzun süre önbellekte tutabilmelerini sağlar; bir dosya değiştiğinde etiket de değişir, böylece tarayıcılar öğeyi otomatik olarak yeniden yükler.

“Minification”

Birçok üçüncü taraf Django aracı ve paketi, HTML, CSS ve JavaScript dosyalarını “küçültme” olanağı sunar. Bu araçlar, gereksiz boşlukları, satır sonlarını ve yorumları kaldırır, değişken adlarını kısaltır ve böylece sitenizin yayınladığı belgelerin boyutunu azaltır.

Şablon performansı

Şunu unutmayın:

- **{% block %}** kullanımı, **{% include %}** kullanımından daha hızlıdır
- çok sayıda küçük parçadan bir araya getirilmiş, aşırı parçalanmış şablonlar performansı olumsuz etkileyebilir

Önbelleğe alınmış şablon yükleyici

cached template loader’ı, etkinleştirmek, şablonların her görüntülenişinde yeniden derlenmesini önlediği için genellikle performansı önemli ölçüde artırır.

Mevcut yazılımın farklı sürümlerini kullanma

Kullandığınız yazılımın farklı ve daha iyi performans gösteren sürümlerinin mevcut olup olmadığını kontrol etmek bazen faydalı olabilir.

Bu teknikler, halihazırda iyi bir şekilde optimize edilmiş bir Django sitesinin performans sınırlarını zorlamak isteyen daha ileri düzey kullanıcıları hedeflemektedir.

Ancak bunlar, performans sorunlarına sihirli çözümler değildir ve temel gereklilikleri doğru bir şekilde yerine getirmeyen sitelerde, ancak çok az bir kazanç sağlamaları muhtemeldir.

Not

Tekrarlamakta fayda var: Halihazırda kullandığınız yazılımlara alternatif aramak, performans sorunlarına karşı asla ilk çözüm değildir. Bu düzeyde bir optimizasyona ulaştığınızda, resmi bir performans karşılaştırma çözümüne ihtiyacınız olacaktır.

Daha yeni olan genellikle - ama her zaman deęil - daha iyidir

İyi bir şekilde bakımı yapılan bir yazılımın yeni sürümünün daha az verimli olması oldukça nadirdir, ancak bakım ekibi her olası kullanım senaryosunu öngöremez; bu nedenle, yeni sürümlerin muhtemelen daha iyi performans göstereceğini bilerek de olsa, her zaman böyle olacağını varsaymayın.

Bu durum Django için de geçerlidir. Ardışık sürümler sistem genelinde bir dizi iyileştirme sunmuştur, ancak yine de uygulamanızın gerçek dünyadaki performansını kontrol etmelisiniz, çünkü bazı durumlarda değişikliklerin performansın iyileşmesi yerine kötüleşmesine neden olduğunu görebilirsiniz.

Python'un ve Python paketlerinin yeni sürümleri de genellikle daha iyi performans gösterir; ancak varsaymak yerine ölçün.

Not

Belirli bir sürümde olağandışı bir performans sorunu yaşamadığınız sürece, yeni sürümlerde genellikle daha iyi özellikler, güvenilirlik ve güvenlik bulacaksınız; üstelik bu avantajlar, elde edebileceğiniz veya kaybedebileceğiniz herhangi bir performans farkından çok daha önemlidir.

Django'nun şablon diline alternatifler

Neredeyse her durumda, Django'nun yerleşik şablon dili gayet yeterlidir. Ancak, Django projenizdeki darboğazların şablon sisteminden kaynaklandığı görülüyorsa ve bunu gidermek için diğer tüm seçenekleri denediyseniz, üçüncü taraf bir alternatif çözüm olabilir.

Jinja2, özellikle hız açısından performans artışı sağlayabilir.

Alternatif şablon sistemleri, Django'nun şablon dilini ne ölçüde paylaştıklarına göre farklılık gösterir.

Not

Eğer şablonlarda performans sorunları yaşarsınız, yapılacak ilk şey nedenini tam olarak anlamaktır. Alternatif bir şablon sistemi kullanmak olabilir daha hızlı kanıtlayın, ancak aynı kazanımlar gitmeden de elde edilebilir bu sorun, örneğin pahalı işlem ve mantıktır şablonlar görünümlerinizde daha verimli bir şekilde yapılabilir.

Alternatif yazılım uygulamaları

Kullandığınız Python yazılımının, aynı kodu daha hızlı çalıştırabilen farklı bir sürümde sunulup sunulmadığını kontrol etmek faydalı olabilir.

Ancak: İyi yazılmış Django sitelerindeki performans sorunlarının çoğu, Python'un çalışma düzeyinde değil, verimsiz veritabanı sorguları, önbellekleme ve şablonlarda yatmaktadır. Kötü yazılmış Python koduna güveniyorsanız, kodun daha hızlı çalıştırılmasıyla performans sorunlarınızın çözülmesi pek olası değildir.

Alternatif bir uygulama kullanmak, uyumluluk, dağıtım, taşınabilirlik veya bakım sorunlarına yol açabilir. Standart olmayan bir uygulamayı benimsemeden önce, uygulamanız için potansiyel risklerden daha fazla fayda sağlayacak yeterli performans artışı sağladığından emin olmanız gerektiği açıktır.

Bu uyarıları göz önünde bulundurarak, şunlara dikkat etmelisiniz:

PyPy

PyPy, Python dilinin Python ile yazılmış bir uygulamasıdır (standart Python uygulaması C dilinde yazılmıştır). PyPy, özellikle kaynak yoğun uygulamalarda önemli ölçüde performans artışı sağlayabilir.

PyPy projesinin temel amaçlarından biri, mevcut Python API'leri ve kütüphaneleriyle uyumluluk sağlamaktır. Django, desteklenen Python sürümlerine karşılık gelen PyPy sürümleriyle uyumludur; ancak kullandığınız diğer kütüphanelerin uyumluluğunu kontrol etmeniz gerekecektir.

Python kütüphanelerinin C dillerinde yazılmış sürümleri

Bazı Python kütüphaneleri C dilinde de yazılmıştır ve çok daha hızlı olabilirler. Bu kütüphanelerin amacı, aynı API'leri sunmaktır. Uyumluluk sorunları ve davranış farklılıklarının yaşanabileceği unutulmamalıdır (ve bunlar her zaman hemen fark edilmeyebilir).